Appendix E – Selected VB.NET Source Code / Repository Submission Details

Repository: https://github.com/CoffeeIsThere/dauntless-wolves-re-re

Reviewed branch: main | Commit: a2037141aa3bfd7da30c50218e4ef6c553eed691 | Retrieved: 2 November 2026

Project: DAUNTLESS WOLVES PROF.sln | VB.NET, .NET 8 Windows Forms, MySQL/MariaDB. The listings below are selected excerpts, not standalone programs. Helpers and supporting methods remain in the repository. Read each listing down the left column, then down the right. Visual wrapping is for documentation only.

Code E1: GradeCalculator.vb – Shared grade calculation

Source: Services/GradeCalculator.vb, lines 1–59.

Computes assessment percentages and weighted running grades. Ungraded items are excluded; recorded zeros count; no running grade is Not Yet.

```
' Pure grade math. No database, so it is easy to test.
Public Class GradeCalculator

    ' Passing mark: the ONE place to change it. Forms should use
      this constant instead of typing 75.
    Public Const PassingGrade As Double = 75.0

    Public Shared Function CalculateScorePercentage(score As
      Double, totalScore As Double) As Double
        If totalScore <= 0 Then Return 0.0
        Return Math.Round((score / totalScore) * 100, 2,
          MidpointRounding.AwayFromZero)
    End Function

    ' Kept for older callers. Statuses are worked out when
      displaying; do not store them in the database.
    Public Shared Function DetermineRemarks(percentage As
      Double) As String
        Return If(percentage >= PassingGrade, "Passed", "Needs
          Improvement")
    End Function

    ' Status of ONE assessment: Pending when not graded yet
    Public Shared Function GetScoreStatus(score As Double?,
      totalScore As Double) As String
        If Not score.HasValue Then Return "Pending"
        Return If(CalculateScorePercentage(score.Value,
          totalScore) >= PassingGrade, "Passed", "Needs
          Improvement")
    End Function

    ' Weighted running grade over graded items only:
    '   sum(score / total x weight) / sum(weight of graded
      items) x 100
    ' Nothing graded yet = Nothing (NULL), never 0.
    Public Shared Function CalculateRunningGrade(items As
      IEnumerable(Of ScoreItem)) As Double?
        Dim earned As Double = 0.0
        Dim weightUsed As Double = 0.0

        For Each it As ScoreItem In items
            If it.Score.HasValue AndAlso it.TotalScore > 0
              AndAlso it.Weight > 0 Then
                earned += (it.Score.Value / it.TotalScore) *
                  it.Weight
                weightUsed += it.Weight
            End If
        Next

        If weightUsed = 0 Then Return Nothing
        Return Math.Round(earned / weightUsed * 100, 2,
          MidpointRounding.AwayFromZero)
    End Function

    Public Shared Function GetStatus(runningGrade As Double?) As
      String
        If Not runningGrade.HasValue Then Return "Not Yet"
        Return If(runningGrade.Value >= PassingGrade, "Passed",
          "At risk")
    End Function

End Class

' One graded (or not yet graded) item, used by the calculator
Public Class ScoreItem
    Public ReadOnly Property Score As Double?
    Public ReadOnly Property TotalScore As Double
    Public ReadOnly Property Weight As Double

    Public Sub New(score As Double?, totalScore As Double,
      weight As Double)
        Me.Score = score
        Me.TotalScore = totalScore
        Me.Weight = weight
    End Sub
End Class
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/GradeCalculator.vb

Code E2: AuthService.vb – Role-aware authentication

Source: Services/AuthService.vb, lines 7–51.

Looks up eligible accounts with a bound login parameter, verifies the password hash and upgrades supported older hashes after successful login.

```
Public Function VerifyLogin(login As String, rawPassword As
    String, Optional preferredRole As String = Nothing) As
    UserModel
    Dim cleanLogin As String =
        ValidationHelper.CleanInput(login)
    If cleanLogin = "" OrElse
        String.IsNullOrEmpty(rawPassword) Then Return
        Nothing

    ' Cast IDs to CHAR and fetch department/program under
        'department_or_program'
    Dim sql As String =
        "SELECT 'Admin' AS Role, CAST(admin_id AS CHAR) AS
            UserId, full_name AS FullName, email,
            password_hash, '' AS department_or_program " &
        "FROM admin WHERE email = @Login " &
        "UNION ALL " &
        "SELECT 'Teacher', CAST(teacher_id AS CHAR),
            full_name, email, password_hash, department " &
        "FROM teacher WHERE (email = @Login OR teacher_id =
            @Login) AND status = 'Active' " &
        "UNION ALL " &
        "SELECT 'Student', CAST(student_id AS CHAR),
            full_name, email, password_hash, program " &
        "FROM student WHERE (email = @Login OR student_id =
            @Login) AND status = 'Active'"

    Dim params As MySqlParameter() = {New
        MySqlParameter("@Login", cleanLogin)}
    Dim dt As DataTable = DatabaseHelper.ExecuteQuery(sql,
        params)

    For Each row As DataRow In
        dt.AsEnumerable().OrderByDescending(Function(r)
        String.Equals(r("Role").ToString(), preferredRole,
        StringComparison.OrdinalIgnoreCase))
        If preferredRole IsNot Nothing AndAlso
            row("Role").ToString() <> "Admin" AndAlso Not
            row("Role").ToString().Equals(preferredRole,
            StringComparison.OrdinalIgnoreCase) Then
            Continue For
        Dim storedHash As String =
            row("password_hash").ToString()

        If SecurityHelper.VerifyPassword(rawPassword,
            storedHash) Then
            Dim role As String = row("Role").ToString()
            Dim userId As String = row("UserId").ToString()

            ' Old SHA-256 hashes are quietly upgraded to the
            salted format after a good login
            If SecurityHelper.NeedsRehash(storedHash) Then
                UpgradeHash(role, userId, rawPassword)
            End If

            Dim deptOrProg As String = If(Not
            IsDBNull(row("department_or_program")),
            row("department_or_program").ToString(), "")

            Return New UserModel With {
                .UserId = userId,
                .FullName = row("FullName").ToString(),
                .Email = row("email").ToString(),
                .Role = role,
                .DepartmentOrProgram = deptOrProg
            }
        End If
    Next

    Return Nothing
End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/AuthService.vb

Code E3: GradeService.vb – Transactional score saving

Source: Services/GradeService.vb, lines 12–32.

Wraps the shared score-saving operation in a database transaction. RecordScoreTx performs the related authorization, validation, grade update and audit work.

```
Public Function RecordScore(enrollmentId As Integer,
  assessmentId As Integer,
                newScore As Double, teacherId As
      String,
                Optional reason As String =
      Nothing) As OperationResult
  Try
    Using conn As MySqlConnection =
      DatabaseHelper.GetConnection()
      Using tx As MySqlTransaction =
      conn.BeginTransaction()
        Try
          Dim res As OperationResult =
      RecordScoreTx(conn, tx, enrollmentId,
      assessmentId, newScore, teacherId, reason)
          If res.Success Then tx.Commit() Else
            tx.Rollback()
            Return res
          Catch
            tx.Rollback()
            Throw
          End Try
        End Using
      End Using
    Catch ex As Exception
      Console.WriteLine("[RECORD SCORE ERROR] " &
        ex.Message)
      Return OperationResult.Fail("Could not save the
        score: " & ex.Message)
    End Try
  End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/GradeService.vb

Code E4: RevisionRequestService.vb – Revision approval and score correction

Source: Services/RevisionRequestService.vb, lines 276–315.

Checks the active assigned teacher and open request, determines the corrected score, and updates the decision and grade within the caller’s transaction.

```
Private Function ApproveTx(conn As MySqlConnection, tx As
    MySqlTransaction,
                requestId As Integer, teacherId
        As String,
                overrideScore As Double?,
        reviewerResponse As String) As OperationResult
    If Not AccountAccess.IsActive(conn, tx, "Teacher",
      teacherId) Then Return OperationResult.Fail("Your
      professor account is inactive.")
    Dim info As RequestInfo = LoadRequest(conn, tx,
      requestId)
    Dim check As OperationResult = CheckOpenAndOwned(info,
      teacherId)
    If Not check.Success Then Return check

    Dim target As Double
    If overrideScore.HasValue Then
      target = overrideScore.Value
    ElseIf info.RequestedPoints.HasValue AndAlso
      info.CurrentScore.HasValue Then
      target = Math.Min(info.CurrentScore.Value +
        info.RequestedPoints.Value, info.TotalScore)
    Else
      Return OperationResult.Fail("NEEDSCORE")
    End If

    ' Mark the request approved (only if it is still open)
    Dim rows As Integer
    Dim updateSql As String =
      "UPDATE revisionrequest SET status = 'Approved',
        reviewed_at = NOW(), reviewed_by = @TeacherId "
        &
      "WHERE request_id = @RequestId AND status IN
          ('Pending','Under Review')"
    Using cmd As New MySqlCommand(updateSql, conn, tx)
      cmd.Parameters.AddWithValue("@TeacherId", teacherId)
      cmd.Parameters.AddWithValue("@RequestId", requestId)
      rows = cmd.ExecuteNonQuery()
    End Using
    If rows <= 0 Then Return OperationResult.Fail("The
      request could not be updated.")

    ' Apply the score through the same code path as normal
      grading (ownership, range, running grade, audit)
    Dim gradeRes As OperationResult =
      _gradeService.RecordScoreTx(conn, tx,
      info.EnrollmentId, info.AssessmentId, target,
      teacherId,
  "Approved revision request #" & requestId.ToString())
    If Not gradeRes.Success Then Return gradeRes

    ' Log the decision itself too (the score change was
      logged by RecordScoreTx)
    AuditService.LogActionTx(conn, tx, "Teacher", teacherId,
      "Updated request status", "RevisionRequest",
                requestId.ToString(),
        info.Status, StatusApproved, "Score set to " &
        Fmt(target) & "/" & Fmt(info.TotalScore) &
        If(String.IsNullOrWhiteSpace(reviewerResponse),
        "", vbCrLf & reviewerResponse.Trim()))

    Return OperationResult.Ok("Request #" &
      requestId.ToString() & " approved. Score is now " &
      Fmt(target) & "/" & Fmt(info.TotalScore) & ".")
  End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/RevisionRequestService.vb

Code E5: DatabaseHelper.vb – Parameterized database query

Source: Core/DatabaseHelper.vb, lines 75–88.

Uses the configured connection and supplied parameters to execute a query and return its rows as a DataTable.

```
    Public Shared Function ExecuteQuery(sql As String,
        ParamArray params As MySqlParameter()) As DataTable
        Dim dt As New DataTable()
        Using conn As MySqlConnection = GetConnection()
            Using cmd As New MySqlCommand(sql, conn)
                If params IsNot Nothing AndAlso params.Length >
                0 Then
                    cmd.Parameters.AddRange(params)
                End If
                Using adapter As New MySqlDataAdapter(cmd)
                    adapter.Fill(dt)
                End Using
            End Using
        End Using
        Return dt
    End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Core/DatabaseHelper.vb

Code E6: StudentProfileService.vb – Student-scoped profile summary

Source: Services/StudentProfileService.vb, lines 6–37.

Reads the logged-in student's profile, class grades and open-request count without changing academic records. Uses the shared grade calculator.

```
Public Function LoadCurrent() As StudentProfileSnapshot
    If Not AppSession.IsStudent Then Throw New
        UnauthorizedAccessException("Student access is
        required.")
    Dim studentId = AppSession.CurrentUser.UserId
    Using conn = DatabaseHelper.GetConnection()
        Using tx =
            conn.BeginTransaction(IsolationLevel.RepeatableR
            ead)
            Dim profile = ReadTable(conn, tx, "SELECT
            student_id,full_name,email,program,year_level,se
            ction,status FROM student WHERE
            student_id=@Student", studentId)
            If profile.Rows.Count <> 1 Then Throw New
            InvalidOperationException("Student profile not
            found.")
            Dim grades = ReadTable(conn, tx,
                "SELECT e.enrollment_id,c.course_code,co.cou
            rse_name,c.section,c.semester,c.academic_year,sg
            .score,a.total_score,a.weight " &
                "FROM enrollment e JOIN class c ON
            c.class_id=e.class_id JOIN course co ON
            co.course_code=c.course_code " &
                "LEFT JOIN assessment a ON
            a.class_id=c.class_id LEFT JOIN studentgrade sg
            ON sg.enrollment_id=e.enrollment_id AND
            sg.assessment_id=a.assessment_id " &
                "WHERE e.student_id=@Student ORDER BY
            c.course_code,c.academic_year,c.semester,c.secti
            on", studentId)
            Dim requests = ReadTable(conn, tx,
                "SELECT COUNT(*) AS open_count FROM
            revisionrequest r JOIN studentgrade sg ON
            sg.grade_id=r.grade_id " &
                "JOIN enrollment e ON
            e.enrollment_id=sg.enrollment_id WHERE
            e.student_id=@Student AND r.status IN
            ('Pending','Under Review')", studentId)
            Dim classes As New DataTable()
            classes.Columns.Add("Class", GetType(String))
            classes.Columns.Add("Running Grade (%)",
            GetType(Double))
            classes.Columns.Add("Standing", GetType(String))
            For Each group In
            grades.AsEnumerable().GroupBy(Function(row)
            CInt(row("enrollment_id")))
                Dim first = group.First()
                Dim items = group.Where(Function(row) Not
            IsDBNull(row("total_score")) AndAlso Not
            IsDBNull(row("weight"))).
                    Select(Function(row) New
            ScoreItem(If(IsDBNull(row("score")),
            CType(Nothing, Double?), CDbl(row("score"))),
            CDbl(row("total_score")), CDbl(row("weight"))))
                Dim grade =
            GradeCalculator.CalculateRunningGrade(items)
                Dim label = first("course_code").ToString()
            & " — " & first("course_name").ToString() & " |
            " & first("section").ToString() & " | " &
            first("semester").ToString() & " | " &
            first("academic_year").ToString()
                classes.Rows.Add(label, If(grade.HasValue,
            CType(grade.GetValueOrDefault(), Object),
            DBNull.Value), GradeCalculator.GetStatus(grade))
            Next
            tx.Commit()
            Return New StudentProfileSnapshot With {.Profile
            = profile.Rows(0), .Classes = classes,
            .OpenRequests =
            CInt(requests.Rows(0)("open_count"))}
        End Using
    End Using
End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/StudentProfileService.vb